SQL Server Interview Handbook

Gated Funnel Format — Candidate + Interviewer Edition

> **How to use:** Start at Gate 1. Stop when the candidate hits their ceiling. Gates 3-4 include actual query challenges.

TABLE OF CONTENTS

GATE 1 — Fundamentals (Junior: 0-3 years)

- Q1. What is the difference between WHERE and HAVING?
- Q2. Explain the different types of JOINs with examples.
- Q3. What is a PRIMARY KEY vs a UNIQUE constraint vs a FOREIGN KEY?
- Q4. What is an index? Why do we need them?
- Q5. What is the difference between DELETE, TRUNCATE, and DROP?
- Q6. What are aggregate functions? Name the common ones.
- Q7. What is the difference between UNION and UNION ALL?
- Q8. What are NULL values and how do you handle them?

GATE 2 — Working Knowledge (Mid: 3-7 years)

- Q9. Explain CTEs (Common Table Expressions). When would you use them?
- Q10. What is the difference between Clustered and Non-Clustered indexes?
- Q11. Stored Procedures vs Functions — when to use which?
- Q12. Scenario: A developer writes this query and it's slow. What do you check?
- Q13. Explain transaction isolation levels.
- Q14. Temp Tables vs Table Variables — when to use which?
- Q15. What is CROSS APPLY vs OUTER APPLY?

GATE 3 — Deep Understanding (Senior: 7-15 years)

- Q16. Explain Window Functions. Write a query using ROW_NUMBER, RANK, and DENSE_RANK.
- Q17. Query Challenge: Find the Nth highest salary.
- Q18. Query Challenge: Find gaps in a sequence (Gaps and Islands).
- Q19. How do you read an execution plan? What are the key things to look for?
- Q20. What is parameter sniffing and how do you handle it?
- Q21. Query Challenge: Running totals and cumulative sums.
- Q22. Query Challenge: Detect and remove duplicates.
- Q23. Scenario: A report query runs in 2 seconds on dev but 45 seconds on production. Same data. What's happening?

GATE 4 — Architecture & Design (Lead/Architect: 15+ years)

- Q24. Scenario: You're designing the database for an e-commerce platform. Walk through your approach.
- Q25. Scenario: A critical table has 500M rows and queries are slow. What's your optimization strategy?
- Q26. How do you handle schema migrations in production?
- Q27. Query Challenge: Pivot data (rows to columns).

APPENDIX — SQL Quick Reference

GATE 1 — Fundamentals `[Junior: 0-3 years]`

> **Style:** Direct questions. Can they write basic SQL and explain core concepts?

Q1. What is the difference between WHERE and HAVING?

Expected Answer:

- `WHERE` filters **rows** before grouping.
- `HAVING` filters **groups** after `GROUP BY`.

```sql
-- WHERE: filter rows before aggregation
SELECT Department, COUNT(*) AS EmpCount
FROM Employees
WHERE Salary > 50000        -- Filter individual rows first
GROUP BY Department;

-- HAVING: filter groups after aggregation
SELECT Department, COUNT(*) AS EmpCount
FROM Employees
GROUP BY Department
HAVING COUNT(*) > 5;         -- Filter groups with more than 5 employees
```

Execution order: `FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY`

Q2. Explain the different types of JOINs with examples.

Expected Answer:

```
Table A          Table B
+----+------+   +----+--------+
| Id | Name |   | Id | Detail |
+----+------+   +----+--------+
|  1 | Alice|   |  1 | X      |
|  2 | Bob  |   |  3 | Y      |
|  3 | Carol|   |  4 | Z      |
+----+------+   +----+--------+
```

INNER JOIN — Only matching rows from both tables:

```
SELECT A.Name, B.Detail
FROM A INNER JOIN B ON A.Id = B.Id;
-- Result: Alice-X, Carol-Y  (Bob excluded: no match, Id=4 excluded: no match)
```

```
A ∩ B:  [Alice-X, Carol-Y]
```

LEFT JOIN — All rows from left + matching from right (NULL if no match):

```
SELECT A.Name, B.Detail
FROM A LEFT JOIN B ON A.Id = B.Id;
-- Result: Alice-X, Bob-NULL, Carol-Y
```

```
All A + matching B:  [Alice-X, Bob-NULL, Carol-Y]
```

RIGHT JOIN — All rows from right + matching from left:

```
SELECT A.Name, B.Detail
FROM A RIGHT JOIN B ON A.Id = B.Id;
-- Result: Alice-X, Carol-Y, NULL-Z
```

FULL OUTER JOIN — All rows from both, NULL where no match:

```
SELECT A.Name, B.Detail
FROM A FULL OUTER JOIN B ON A.Id = B.Id;
-- Result: Alice-X, Bob-NULL, Carol-Y, NULL-Z
```

CROSS JOIN — Cartesian product (every row × every row):

```
SELECT A.Name, B.Detail
FROM A CROSS JOIN B;
-- Result: 3 × 3 = 9 rows (every combination)
```

SELF JOIN — Table joined to itself:

```sql
-- Find employees and their managers
SELECT E.Name AS Employee, M.Name AS Manager
FROM Employees E
INNER JOIN Employees M ON E.ManagerId = M.Id;
```

Q3. What is a PRIMARY KEY vs a UNIQUE constraint vs a FOREIGN KEY?

Expected Answer:

Feature	PRIMARY KEY	UNIQUE	FOREIGN KEY
Uniqueness	Yes	Yes	No
NULLs allowed	No	Yes (one NULL)	Yes
Per table	One only	Multiple	Multiple
Creates index	Clustered (default)	Non-clustered	No index by default
Purpose	Row identity	Alternate unique identifier	Referential integrity

```sql
CREATE TABLE Orders (
    OrderId INT PRIMARY KEY,                    -- Identity
    OrderNumber NVARCHAR(20) UNIQUE,            -- Business key
    CustomerId INT FOREIGN KEY REFERENCES Customers(Id)  -- Relationship
);
```

Q4. What is an index? Why do we need them?

Expected Answer:

An index is a **data structure that speeds up data retrieval** — like a book's index.

Without index: SQL Server scans every row (table scan) — O(n). **With index:** SQL Server jumps directly to matching rows — O(log n).

```sql
-- Create an index
CREATE INDEX IX_Employees_LastName ON Employees(LastName);

-- Now this query uses the index instead of scanning all rows:
SELECT * FROM Employees WHERE LastName = 'Smith';
```

Trade-off: Indexes speed up reads but slow down writes (INSERT, UPDATE, DELETE must also update the index).

Q5. What is the difference between DELETE, TRUNCATE, and DROP?

Expected Answer:

Feature	DELETE	TRUNCATE	DROP
What it removes	Specific rows (with WHERE)	All rows	Entire table
Logged	Fully (row by row)	Minimally (page deallocation)	Fully
WHERE clause	Yes	No	N/A
Identity reset	No	Yes (resets to seed)	N/A
Triggers	Fires DELETE triggers	No triggers	No triggers
Rollback	Yes	Yes (in transaction)	Yes (in transaction)
Speed	Slow (large tables)	Fast	Fast

```sql
DELETE FROM Orders WHERE Status = 'Cancelled';  -- Remove specific rows
TRUNCATE TABLE TempData;                         -- Remove all rows quickly
DROP TABLE OldArchive;                           -- Remove entire table
```

Q6. What are aggregate functions? Name the common ones.

Expected Answer:

```sql
SELECT
    COUNT(*) AS TotalOrders,          -- Count rows
    COUNT(DISTINCT CustomerId) AS UniqueCustomers,  -- Count distinct
    SUM(Amount) AS TotalRevenue,      -- Sum values
    AVG(Amount) AS AverageOrder,      -- Average
    MIN(Amount) AS SmallestOrder,     -- Minimum
    MAX(Amount) AS LargestOrder       -- Maximum
FROM Orders
WHERE OrderDate >= '2024-01-01';
```

Key rule: Aggregate functions ignore NULLs (except `COUNT(*)`).

```sql
-- These are different!
SELECT COUNT(*) FROM Orders;            -- Counts ALL rows (including NULLs)
SELECT COUNT(Discount) FROM Orders;     -- Counts only non-NULL Discount values
```

Q7. What is the difference between UNION and UNION ALL?

Expected Answer:

Feature	UNION	UNION ALL
Duplicates	Removed	Kept
Performance	Slower (sorts to remove duplicates)	Faster
Use when	Need distinct results	Know there are no duplicates, or want all

```
-- UNION: removes duplicates (sorts internally)
SELECT Name FROM Employees
UNION
SELECT Name FROM Contractors;

-- UNION ALL: keeps everything (faster)
SELECT Name FROM Employees
UNION ALL
SELECT Name FROM Contractors;
```

Tip: Always prefer `UNION ALL` unless you specifically need deduplication.

Q8. What are NULL values and how do you handle them?

Expected Answer:

NULL means **unknown/missing** — not zero, not empty string.

```
-- NULL comparisons are tricky
SELECT * FROM Users WHERE MiddleName = NULL;      -- WRONG! Returns nothing
SELECT * FROM Users WHERE MiddleName IS NULL;      -- CORRECT

-- NULL in arithmetic
SELECT 5 + NULL;    -- NULL (anything + NULL = NULL)
SELECT NULL = NULL;  -- NULL (not TRUE!)

-- COALESCE: return first non-NULL
SELECT COALESCE(Nickname, FirstName, 'Unknown') AS DisplayName FROM Users;

-- ISNULL: replace NULL with default (SQL Server specific)
SELECT ISNULL(Phone, 'N/A') FROM Users;

-- NULLIF: return NULL if values are equal
SELECT NULLIF(Quantity, 0) FROM Products;  -- Returns NULL if Quantity = 0
```

↑ Back to Index

GATE 2 — Working Knowledge [Mid: 3-7 years]

> **Style:** Mix of direct + scenario-based. Can they write more complex queries and explain behavior?

Q9. Explain CTEs (Common Table Expressions). When would you use them?

Expected Answer:

A CTE is a **temporary named result set** that exists only for the duration of a single query.

```
-- Non-recursive CTE — readable subqueries
WITH ActiveOrders AS (
    SELECT CustomerId, COUNT(*) AS OrderCount, SUM(Amount) AS TotalSpent
    FROM Orders
    WHERE Status = 'Active'
    GROUP BY CustomerId
)
SELECT C.Name, AO.OrderCount, AO.TotalSpent
FROM Customers C
INNER JOIN ActiveOrders AO ON C.Id = AO.CustomerId
WHERE AO.TotalSpent > 1000;
```

```
-- Recursive CTE — hierarchical data (org chart, categories)
WITH OrgChart AS (
    -- Anchor: top-level managers
    SELECT Id, Name, ManagerId, 0 AS Level
    FROM Employees
    WHERE ManagerId IS NULL

    UNION ALL

    -- Recursive: employees reporting to previous level
    SELECT E.Id, E.Name, E.ManagerId, OC.Level + 1
    FROM Employees E
    INNER JOIN OrgChart OC ON E.ManagerId = OC.Id
)
SELECT * FROM OrgChart ORDER BY Level, Name;
```

CTE vs Temp Table vs Subquery:

Feature	CTE	Temp Table	Subquery
Scope	Single query	Session	Single query
Reusable in query	Yes (reference multiple times)	Yes	No (re-evaluated each time)

Feature	CTE	Temp Table	Subquery
Statistics	No	Yes	No
Best for	Readability, recursion	Large datasets, multiple queries	Simple one-off filtering

---

Q10. What is the difference between Clustered and Non-Clustered indexes?

Expected Answer:

Feature	Clustered Index	Non-Clustered Index
Data order	Physically sorts table data	Separate structure pointing to data
Per table	One only	Up to 999
Leaf level	The actual data rows	Pointers to data rows (or clustered key)
Created by	PRIMARY KEY (default)	CREATE INDEX
Size	Is the table	Additional storage

```
Clustered Index (IS the table):
┌──────────┬──────────┬──────────┐
│ Id (key) │ Name     │ Salary   │  ← Data sorted by Id
├──────────┼──────────┼──────────┤
│ 1        │ Alice    │ 50000    │
│ 2        │ Bob      │ 60000    │
│ 3        │ Carol    │ 55000    │
└──────────┴──────────┴──────────┘

Non-Clustered Index on Name:
┌──────────┬──────────────┐
│ Name     │ Row Pointer  │  ← Separate structure
├──────────┼──────────────┤
│ Alice    │ → Row 1      │
│ Bob      │ → Row 2      │
│ Carol    │ → Row 3      │
└──────────┴──────────────┘
```

Covering index: Includes all columns needed by the query — no need to go back to the table.

```sql
CREATE INDEX IX_Orders_Status
ON Orders(Status)
INCLUDE (OrderDate, Amount);  -- Covers queries needing Status + OrderDate +
Amount
```

Q11. Stored Procedures vs Functions — when to use which?

Expected Answer:

Feature	Stored Procedure	Function
Return value	Optional (OUTPUT params, result sets)	Required (scalar or table)
Side effects	Yes (INSERT, UPDATE, DELETE)	No (read-only, no DML)
Used in SELECT	No	Yes (`SELECT dbo.GetTax(salary)`)
Used in WHERE	No	Yes (`WHERE dbo.IsActive(Id) = 1`)
Error handling	TRY/CATCH	Limited
Transaction control	Yes	No

```sql
-- Stored Procedure: performs actions, returns results
CREATE PROCEDURE GetOrdersByCustomer
    @CustomerId INT,
    @MinAmount DECIMAL(10,2) = 0  -- Default parameter
AS
BEGIN
    SELECT OrderId, Amount, OrderDate
    FROM Orders
    WHERE CustomerId = @CustomerId AND Amount >= @MinAmount
    ORDER BY OrderDate DESC;
END;

EXEC GetOrdersByCustomer @CustomerId = 42, @MinAmount = 100;
```

```sql
-- Scalar Function: computes a value
CREATE FUNCTION dbo.CalculateTax(@Income DECIMAL(10,2))
RETURNS DECIMAL(10,2)
AS
BEGIN
    RETURN @Income * 0.20;
END;

SELECT Name, Salary, dbo.CalculateTax(Salary) AS Tax FROM Employees;
```

```sql
-- Inline Table-Valued Function (best performance for table results)
CREATE FUNCTION dbo.GetActiveOrders(@CustomerId INT)
RETURNS TABLE
AS
RETURN (
    SELECT OrderId, Amount, OrderDate
    FROM Orders
```

```
    WHERE CustomerId = @CustomerId AND Status = 'Active'
);

SELECT * FROM dbo.GetActiveOrders(42);
```

Performance warning: Scalar functions in WHERE clauses cause row-by-row execution. Prefer inline TVFs.

Q12. Scenario: A developer writes this query and it's slow. What do you check?

```
SELECT *
FROM Orders O
INNER JOIN Customers C ON O.CustomerId = C.Id
WHERE YEAR(O.OrderDate) = 2024
ORDER BY O.Amount DESC;
```

Expected Answer (multiple issues):

1. `SELECT *` — Fetches all columns. Select only what's needed.
2. `YEAR(O.OrderDate)` — Function on a column prevents index usage (non-SARGable). Fix:

   ```
   WHERE O.OrderDate >= '2024-01-01' AND O.OrderDate < '2025-01-01'
   ```

3. **Missing index** — Check execution plan. Likely needs index on `OrderDate` or a covering index.
4. **Large result set + ORDER BY** — Sorts can be expensive. Is the sort needed?

SARGable = Search ARGument ABLE — the query optimizer can use an index on that column.

Non-SARGable (BAD)	SARGable (GOOD)
`WHERE YEAR(Date) = 2024`	`WHERE Date >= '2024-01-01'`
`WHERE Name LIKE '%smith'`	`WHERE Name LIKE 'smith%'`
`WHERE ISNULL(Col, 0) > 5`	`WHERE Col > 5`
`WHERE Col * 2 > 100`	`WHERE Col > 50`

Q13. Explain transaction isolation levels.

Expected Answer:

Level	Dirty Read	Non-Repeatable Read	Phantom Read	Concurrency
READ UNCOMMITTED	Yes	Yes	Yes	Highest

Level	Dirty Read	Non-Repeatable Read	Phantom Read	Concurrency
READ COMMITTED (default)	No	Yes	Yes	High
REPEATABLE READ	No	No	Yes	Medium
SERIALIZABLE	No	No	No	Lowest
SNAPSHOT	No	No	No	High (uses versioning)

Definitions:

- **Dirty read:** Reading data that another transaction hasn't committed yet.
- **Non-repeatable read:** Reading the same row twice gets different values (another tx updated it).
- **Phantom read:** Re-running a query gets extra rows (another tx inserted them).

```sql
-- Explicit isolation level
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
BEGIN TRANSACTION;
    SELECT * FROM Accounts WHERE Id = 1;
    -- Other transactions can update/insert other rows
    -- but not change Id=1 until we commit
COMMIT;
```

Q14. Temp Tables vs Table Variables — when to use which?

Expected Answer:

Feature	#TempTable	@TableVariable
Storage	tempdb (disk)	Memory (small), tempdb (large)
Statistics	Yes (optimizer knows row count)	No (assumes 1 row!)
Indexes	Full support	PK + UNIQUE only
Transactions	Participates in ROLLBACK	NOT rolled back
Scope	Session / stored proc	Batch / stored proc
Best for	Large datasets (> ~100 rows)	Small lookups (< ~100 rows)

```sql
-- Temp table: better for large result sets
CREATE TABLE #ActiveUsers (
    UserId INT PRIMARY KEY,
    Name NVARCHAR(100),
    LastLogin DATETIME
```

```
);
INSERT INTO #ActiveUsers
SELECT Id, Name, LastLogin FROM Users WHERE IsActive = 1;
-- Optimizer uses statistics → better execution plans

-- Table variable: better for small, simple lookups
DECLARE @Ids TABLE (Id INT);
INSERT INTO @Ids VALUES (1), (2), (3);
SELECT * FROM Products WHERE CategoryId IN (SELECT Id FROM @Ids);
```

Scenario: "*A developer uses a table variable for 100K rows and the query is slow. Why?*"

Answer: Table variables have no statistics — the optimizer assumes 1 row. This causes terrible plan choices (nested loops instead of hash joins). Switch to a temp table.

Q15. What is CROSS APPLY vs OUTER APPLY?

Expected Answer:

`APPLY` is like a JOIN but calls a **table-valued function** or subquery **for each row**.

```
-- CROSS APPLY: Like INNER JOIN — only rows with results
SELECT C.Name, TopOrder.Amount, TopOrder.OrderDate
FROM Customers C
CROSS APPLY (
    SELECT TOP 3 Amount, OrderDate
    FROM Orders O
    WHERE O.CustomerId = C.Id
    ORDER BY Amount DESC
) TopOrder;
-- Shows each customer's top 3 orders (customers with no orders are excluded)

-- OUTER APPLY: Like LEFT JOIN — all rows, NULL if no results
SELECT C.Name, TopOrder.Amount, TopOrder.OrderDate
FROM Customers C
OUTER APPLY (
    SELECT TOP 3 Amount, OrderDate
    FROM Orders O
    WHERE O.CustomerId = C.Id
    ORDER BY Amount DESC
) TopOrder;
-- Shows all customers, NULL for those with no orders
```

When to use APPLY: When you need to invoke a table-valued function per row, or get "top N per group."

↑ Back to Index

GATE 3 — Deep Understanding [Senior: 7-15 years]

> **Style:** Mostly scenario-based with actual query challenges. Optimization and advanced SQL.

Q16. Explain Window Functions. Write a query using ROW_NUMBER, RANK, and DENSE_RANK.

Expected Answer:

Window functions perform calculations **across a set of rows** related to the current row — without collapsing rows like GROUP BY.

```
SELECT
    Name,
    Department,
    Salary,
    ROW_NUMBER() OVER (ORDER BY Salary DESC) AS RowNum,     -- 1,2,3,4,5 (no gaps, no ties)
    RANK()       OVER (ORDER BY Salary DESC) AS Rank,        -- 1,2,2,4,5 (ties get same rank, gap after)
    DENSE_RANK() OVER (ORDER BY Salary DESC) AS DenseRank    -- 1,2,2,3,4 (ties, no gaps)
FROM Employees;
```

Name	Salary	ROW_NUMBER	RANK	DENSE_RANK
Alice	80000	1	1	1
Bob	70000	2	2	2
Carol	70000	3	2	2
Dave	60000	4	4	3

PARTITION BY — window per group:

```
-- Rank employees within their department
SELECT
    Name, Department, Salary,
    ROW_NUMBER() OVER (PARTITION BY Department ORDER BY Salary DESC) AS DeptRank
FROM Employees;
```

LAG / LEAD — access previous/next row:

```sql
-- Compare each month's sales to the previous month
SELECT
    Month,
    Revenue,
    LAG(Revenue, 1) OVER (ORDER BY Month) AS PrevMonth,
    Revenue - LAG(Revenue, 1) OVER (ORDER BY Month) AS Growth
FROM MonthlySales;
```

Running total:

```sql
SELECT
    OrderDate,
    Amount,
    SUM(Amount) OVER (ORDER BY OrderDate ROWS UNBOUNDED PRECEDING) AS RunningTotal
FROM Orders;
```

Q17. Query Challenge: Find the Nth highest salary.

Expected Answer:

```sql
-- Method 1: DENSE_RANK (handles ties correctly)
WITH RankedSalaries AS (
    SELECT Salary, DENSE_RANK() OVER (ORDER BY Salary DESC) AS Rank
    FROM Employees
)
SELECT DISTINCT Salary
FROM RankedSalaries
WHERE Rank = @N;

-- Method 2: OFFSET-FETCH (simpler, but doesn't handle ties)
SELECT DISTINCT Salary
FROM Employees
ORDER BY Salary DESC
OFFSET (@N - 1) ROWS
FETCH NEXT 1 ROWS ONLY;

-- Method 3: Subquery (classic interview answer)
SELECT MAX(Salary) AS NthHighest
FROM Employees
WHERE Salary NOT IN (
    SELECT TOP (@N - 1) Salary
    FROM Employees
    ORDER BY Salary DESC
);
```

What to listen for: Handles ties (DENSE_RANK). Knows OFFSET-FETCH. Considers edge cases (what if fewer than N distinct salaries?).

Q18. Query Challenge: Find gaps in a sequence (Gaps and Islands).

Scenario: "*We have a table of logged-in users with consecutive session IDs, but some IDs are missing. Find the gaps.*"

```sql
-- Sample: Sessions table has Ids: 1,2,3,7,8,12,13,14,15
-- Expected gaps: 4-6, 9-11

WITH Gaps AS (
    SELECT
        Id AS CurrentId,
        LEAD(Id) OVER (ORDER BY Id) AS NextId
    FROM Sessions
)
SELECT
    CurrentId + 1 AS GapStart,
    NextId - 1 AS GapEnd
FROM Gaps
WHERE NextId - CurrentId > 1;
```

Islands (consecutive ranges):

```sql
-- Find consecutive ranges: 1-3, 7-8, 12-15
WITH Islands AS (
    SELECT
        Id,
        Id - ROW_NUMBER() OVER (ORDER BY Id) AS GroupId
    FROM Sessions
)
SELECT
    MIN(Id) AS IslandStart,
    MAX(Id) AS IslandEnd,
    COUNT(*) AS IslandSize
FROM Islands
GROUP BY GroupId;
```

Q19. How do you read an execution plan? What are the key things to look for?

Expected Answer:

```sql
-- Generate actual execution plan
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
```

```
-- Or in SSMS: Ctrl+M (include actual execution plan), then run query
```

Key operators to look for:

Operator	Good/Bad	Meaning
Index Seek	Good	Directly finding rows via index
Index Scan	Usually bad	Reading entire index
Table Scan	Bad	Reading entire table (no index)
Key Lookup	Bad if frequent	Index found rows but needs additional columns from table
Hash Match	Can be OK	Hash join for large datasets
Nested Loops	Good for small sets	Bad for large sets (row-by-row)
Sort	Expensive	Sorting data (consider adding ordered index)
Parallelism	Context-dependent	Query split across CPUs

What to check:

1. **Estimated vs Actual rows** — Big difference = stale statistics.
2. **Fat arrows** — Thick data flow arrows = lots of data being processed.
3. **Warnings** — Yellow triangles (implicit conversion, missing index hints).
4. **Cost percentage** — Which operator takes the most time?

Scenario: "*Execution plan shows Key Lookup (80% cost). How do you fix it?*"

Answer: Add a covering index with `INCLUDE` for the columns being looked up:

```
-- Before: Index Seek on Status, then Key Lookup for Amount and OrderDate
CREATE INDEX IX_Orders_Status ON Orders(Status)
INCLUDE (Amount, OrderDate);
-- After: Index Seek covers everything — no Key Lookup needed
```

Q20. What is parameter sniffing and how do you handle it?

Expected Answer:

SQL Server compiles a query plan based on the **first parameter values** it sees. If those values are atypical, the plan may be bad for subsequent calls.

```
-- This stored proc is called first with @Status = 'Active' (1M rows)
-- Plan: Table Scan (good for 1M rows)
-- Then called with @Status = 'VIP' (10 rows)
-- Plan is still Table Scan — terrible for 10 rows! Should be Index Seek.
```

```sql
CREATE PROCEDURE GetOrdersByStatus @Status NVARCHAR(20)
AS
    SELECT * FROM Orders WHERE Status = @Status;
```

Solutions:

```sql
-- 1. OPTIMIZE FOR UNKNOWN (uses average statistics)
CREATE PROCEDURE GetOrdersByStatus @Status NVARCHAR(20)
AS
    SELECT * FROM Orders WHERE Status = @Status
    OPTION (OPTIMIZE FOR UNKNOWN);

-- 2. RECOMPILE (new plan every time — overhead but always optimal)
CREATE PROCEDURE GetOrdersByStatus @Status NVARCHAR(20)
AS
    SELECT * FROM Orders WHERE Status = @Status
    OPTION (RECOMPILE);

-- 3. Local variable (breaks sniffing — uses average stats)
CREATE PROCEDURE GetOrdersByStatus @Status NVARCHAR(20)
AS
BEGIN
    DECLARE @LocalStatus NVARCHAR(20) = @Status;
    SELECT * FROM Orders WHERE Status = @LocalStatus;
END;
```

Q21. Query Challenge: Running totals and cumulative sums.

```sql
-- Monthly revenue with running total and percentage of yearly total
WITH MonthlyData AS (
    SELECT
        MONTH(OrderDate) AS Month,
        SUM(Amount) AS Revenue
    FROM Orders
    WHERE YEAR(OrderDate) = 2024
    GROUP BY MONTH(OrderDate)
)
SELECT
    Month,
    Revenue,
    SUM(Revenue) OVER (ORDER BY Month ROWS UNBOUNDED PRECEDING) AS RunningTotal,
    CAST(Revenue AS DECIMAL(10,2)) / SUM(Revenue) OVER () * 100 AS PctOfTotal,
    Revenue - LAG(Revenue) OVER (ORDER BY Month) AS MoMChange
FROM MonthlyData
ORDER BY Month;
```

Q22. Query Challenge: Detect and remove duplicates.

Expected Answer:

```
-- Find duplicates
SELECT Email, COUNT(*) AS DuplicateCount
FROM Users
GROUP BY Email
HAVING COUNT(*) > 1;

-- Remove duplicates, keeping the latest record
WITH Ranked AS (
    SELECT *,
        ROW_NUMBER() OVER (PARTITION BY Email ORDER BY CreatedDate DESC) AS RowNum
    FROM Users
)
DELETE FROM Ranked WHERE RowNum > 1;
-- Keeps RowNum = 1 (latest) for each Email group
```

Q23. Scenario: A report query runs in 2 seconds on dev but 45 seconds on production. Same data. What's happening?

Expected Answer (diagnostic approach):

1. **Different execution plans** — Parameter sniffing may have cached a bad plan.

   ```
   -- Check cached plan
   SELECT * FROM sys.dm_exec_query_stats
   CROSS APPLY sys.dm_exec_sql_text(sql_handle)
   WHERE text LIKE '%YourQuery%';
   ```

2. **Stale statistics** — Production data may have changed significantly.

   ```
   UPDATE STATISTICS TableName WITH FULLSCAN;
   ```

3. **Blocking** — Other queries holding locks in production.

   ```
   SELECT * FROM sys.dm_exec_requests WHERE blocking_session_id > 0;
   ```

4. **Hardware differences** — Memory, CPU, disk speed differ.

5. **tempdb contention** — Heavy temp table usage in production.

6. **Different indexes** — Dev may have different index setup.

↑ Back to Index

GATE 4 — Architecture & Design `[Lead/Architect: 15+ years]`

> **Style:** Database design decisions, optimization strategy, scaling.

Q24. Scenario: You're designing the database for an e-commerce platform. Walk through your approach.

Expected Answer (should cover):

Core tables:

```
Customers (Id, Name, Email, CreatedDate)
Products (Id, Name, Price, CategoryId, StockQuantity)
Orders (Id, CustomerId, OrderDate, Status, TotalAmount)
OrderItems (Id, OrderId, ProductId, Quantity, UnitPrice, LineTotal)
Categories (Id, Name, ParentCategoryId)  -- Self-referencing for hierarchy
```

Key decisions:

1. **Denormalization for read performance:** Store `TotalAmount` on Orders (calculated field) to avoid summing OrderItems on every read.
2. **Soft deletes:** `IsDeleted` flag instead of physical delete (audit trail, recoverability).
3. **Indexing strategy:**
   - Clustered on `Id` (identity).
   - Non-clustered on `Orders(CustomerId, OrderDate)` — common query pattern.
   - Covering index on `OrderItems(OrderId) INCLUDE (ProductId, Quantity, UnitPrice)`.
4. **Partitioning:** `Orders` table partitioned by year for archival performance.

Q25. Scenario: A critical table has 500M rows and queries are slow. What's your optimization strategy?

Expected Answer (layered approach):

1. **Query optimization first:**
   - Review execution plans for the slowest queries.
   - Fix non-SARGable predicates.
   - Add/modify indexes based on actual query patterns.
2. **Indexing strategy:**
   - Identify top 10 queries by CPU/IO (from query store or DMVs).

    - Create covering indexes to eliminate key lookups.
    - Remove unused indexes (they slow down writes).

3. **Partitioning:**
    - Partition by date range if queries are time-bounded.
    - Partition elimination means only relevant partitions are scanned.

4. **Archiving:**
    - Move old data to archive table/database.
    - Keep hot data small.

5. **Caching layer:**
    - Redis for frequently accessed read data.
    - Application-level caching for reference data.

6. **Read replicas:**
    - Offload reporting queries to a read replica.
    - Keep writes on primary.

What to listen for: Starts with query optimization (cheapest), not infrastructure changes (expensive).

Q26. How do you handle schema migrations in production?

Expected Answer:

Principles:

1. **Backward compatible:** Never make breaking changes in one step.
2. **Zero downtime:** Migrations must work while the app is running.
3. **Rollback plan:** Every migration has a reverse script.

Adding a column (safe):

```sql
-- Step 1: Add column with default (no lock on modern SQL Server)
ALTER TABLE Orders ADD Priority INT DEFAULT 0;

-- Step 2: Backfill data (batched to avoid locks)
WHILE 1 = 1
BEGIN
    UPDATE TOP (10000) Orders
    SET Priority = CASE WHEN Amount > 1000 THEN 1 ELSE 0 END
    WHERE Priority IS NULL;

    IF @@ROWCOUNT = 0 BREAK;
END;

-- Step 3: Make NOT NULL after backfill
ALTER TABLE Orders ALTER COLUMN Priority INT NOT NULL;
```

Renaming a column (requires expand-contract):

```
-- Deploy 1: Add new column, keep old column
ALTER TABLE Orders ADD NewColumnName INT;

-- Backfill + app writes to both columns

-- Deploy 2: App reads from new column only

-- Deploy 3: Drop old column
ALTER TABLE Orders DROP COLUMN OldColumnName;
```

Q27. Query Challenge: Pivot data (rows to columns).

Scenario: "*Transform monthly sales data from rows into a cross-tab report.*"

```
-- Source data:
-- | Year | Month | Revenue |
-- | 2024 | Jan   | 10000   |
-- | 2024 | Feb   | 12000   |
-- | 2024 | Mar   | 15000   |

-- Desired output:
-- | Year | Jan   | Feb   | Mar   |
-- | 2024 | 10000 | 12000 | 15000 |

SELECT Year, [Jan], [Feb], [Mar], [Apr], [May], [Jun],
             [Jul], [Aug], [Sep], [Oct], [Nov], [Dec]
FROM (
    SELECT Year, Month, Revenue FROM MonthlySales
) AS Source
PIVOT (
    SUM(Revenue)
    FOR Month IN ([Jan],[Feb],[Mar],[Apr],[May],[Jun],
                  [Jul],[Aug],[Sep],[Oct],[Nov],[Dec])
) AS PivotTable;
```

⬆ Back to Index

APPENDIX — SQL Quick Reference

String Functions

```
LEN('Hello')          -- 5
LEFT('Hello', 3)      -- 'Hel'
RIGHT('Hello', 3)     -- 'llo'
SUBSTRING('Hello', 2, 3)  -- 'ell'
CHARINDEX('l', 'Hello')   -- 3
REPLACE('Hello', 'l', 'L') -- 'HeLLo'
LTRIM(RTRIM('  Hi  '))    -- 'Hi'
UPPER('hello')        -- 'HELLO'
STRING_AGG(Name, ', ') -- 'Alice, Bob, Carol' (SQL 2017+)
```

Date Functions

```
GETDATE()             -- Current datetime
GETUTCDATE()          -- Current UTC datetime
DATEADD(DAY, 7, date) -- Add 7 days
DATEDIFF(DAY, d1, d2) -- Days between dates
FORMAT(date, 'yyyy-MM-dd') -- Format date
YEAR(date), MONTH(date), DAY(date)
EOMONTH(date)         -- Last day of month
DATEPART(WEEKDAY, date)  -- Day of week
```

Useful Patterns

```
-- Pagination
SELECT * FROM Products
ORDER BY Name
OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY;  -- Page 3 (10 per page)

-- Upsert (MERGE)
MERGE INTO Target AS T
USING Source AS S ON T.Id = S.Id
WHEN MATCHED THEN UPDATE SET T.Name = S.Name
WHEN NOT MATCHED THEN INSERT (Id, Name) VALUES (S.Id, S.Name);

-- Conditional aggregation
SELECT
    COUNT(CASE WHEN Status = 'Active' THEN 1 END) AS ActiveCount,
    COUNT(CASE WHEN Status = 'Inactive' THEN 1 END) AS InactiveCount
FROM Users;

-- Top N per group
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY Department ORDER BY Salary DESC) AS
RN
    FROM Employees
) T WHERE RN <= 3;
```

> **Gate Assessment Summary:**
>
> | Gate | Topic Focus | Passed → Level |
> |---|---|---|
> | Gate 1 | JOINs, WHERE/HAVING, indexes, NULL handling, basic SQL | Junior |
> | Gate 2 | CTEs, index types, stored procs, SARGability, isolation levels | Mid |
> | Gate 3 | Window functions, execution plans, parameter sniffing, query challenges | Senior |
> | Gate 4 | Database design, optimization strategy, schema migrations, advanced patterns | Lead/Architect |

⬆ Back to Index